GIT & GITHUB WORKSHOP

Student Handout | ACM JIT

1 Git vs GitHub

Git	GitHub
A version control tool that runs on your computer.	An online platform for hosting and collaborating on Git repositories.

REMEMBER Git tracks your work. GitHub helps you share and collaborate on that work.

2 Before You Start

- Make sure Git is installed: git --version
- Have a GitHub account ready.
- Use a project folder you can safely experiment with.

```
git --version
```

3 Essential Git Commands

Purpose	Command	What it means
Start tracking a project	git init	Creates a Git repository in the current folder.
Check what changed	git status	Shows modified, staged, and untracked files.
Stage changes	git add .	Moves current changes into the staging area.
Save a checkpoint	git commit -m "message"	Creates a commit in Git history.
View history	git log --oneline	Shows a compact commit history.

CORE WORKFLOW Working files -> git add -> staging area -> git commit -> Git history

4 Your First Local Project

```
mkdir git-workshop
cd git-workshop
mkdir my-project
cd my-project
git init
git status
```

TRY IT: Create a simple file such as index.html, add some text, then run git status, git add ., and git commit.

5 GitHub Account & Configuration

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global --list
```

WHY? Git uses this identity when creating commits.

6 Connect Your Project to GitHub

```
git remote add origin YOUR_REPOSITORY_URL
git remote -v
git push -u origin main
```

CHECKPOINT Refresh your GitHub repository and confirm that your project files are visible.

7 Clone & Pull

```
git clone YOUR_REPOSITORY_URL
git pull
```

- git clone = bring an existing GitHub repository onto your computer.
- git pull = bring the latest remote changes into your local project.

8 Branches & Collaboration

```
git switch -c feature/about-page
git add .
git commit -m "Add about page"
git push -u origin feature/about-page
```

TYPICAL IDEA main -> feature branch -> changes -> push -> Pull Request -> review -> merge.

9 README & GitHub Profile

- README.md explains what a repository is, how to use it, and what the project contains.
- A GitHub profile README can introduce you, your skills, and your projects.

```
# My Git Workshop

This is my first Git and GitHub project.

## What I learned
- Git
- GitHub
- Commits
- Branches
- Collaboration
```

10 .gitignore & Security

```
node_modules/
.env
dist/
build/
```

NEVER PUSH: passwords, API keys, database credentials, private tokens, or secret .env files.

SECURITY RULE Treat GitHub as public unless you deliberately understand the repository visibility and the data you are uploading.

11 Undoing Mistakes

Situation	Command	Purpose
Unstage changes	git restore --staged .	Removes files from the staging area.
Discard an uncommitted file change	git restore filename	Restores the file to its last committed version.
Undo a commit safely	git revert HEAD	Creates a new commit that reverses the latest commit.

12 Final Practice Challenge

- Create a GitHub repository named my-first-git-project.
- Create or add a small project file.
- Initialize Git and make your first commit.
- Connect the project to GitHub and push it.
- Create a feature branch.
- Make one change and commit it.
- Push the feature branch.
- Open a Pull Request and understand what it is used for.

FINISH LINE Your goal: leave with one GitHub repository that you created and can explain.

Quick Command Sheet

KEEP THIS PAGE AS YOUR QUICK REFERENCE

```
git --version
git init
git status
git add .
git commit -m "message"
git log --oneline
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

```
git remote -v
git remote add origin URL
git push -u origin main
git pull
git clone URL
git switch -c feature-name
git restore filename
git restore --staged .
git revert HEAD
```

BEFORE YOU LEAVE

[]Git installed[
[]GitHub account ready
[]Project pushed to GitHub
[ ] Feature branch created
[ ] Pull Request opened

3 RULES TO REMEMBER

[]Commit small, meaningful changes
[]Never commit passwords or secrets
[]When collaborating: pull, work, push

Build it. Break it. Fix it. Commit it.